# FALLEN EDEN

Game Design Document + Plan de producción

Versión 0.1 · Documento de trabajo · 1 de noviembre de 2026

RPG isométrico 3D de ciencia ficción oscura, exploración compacta y combate en tiempo real con pausa.

Documento vivo: distingue decisiones acordadas, propuestas y asuntos pendientes. Diseñado para guiar la construcción de un prototipo completo y escalable sin sobrediseñar el proyecto.

# Cómo leer este documento

Las reglas expresadas como principios reflejan la dirección acordada. Los números y listas marcados como provisionales deben validarse con prototipos. Las decisiones abiertas se mantienen visibles para evitar que una sugerencia se convierta accidentalmente en requisito.

# 1. Resumen ejecutivo

Fallen Eden (título provisional) es un RPG de acción isométrico 3D para PC, de ciencia ficción oscura y horror cósmico. Combina exploración de mazmorras, progresión de personaje, botín significativo y combate en tiempo real con pausa. La inspiración toma de Diablo I/II la atmósfera, exploración y progresión; de Neverwinter Nights la lectura táctica; y de Quake, DUSK, Half-Life y Event Horizon el lenguaje visual y el horror, no sus controles FPS.

El jugador selecciona objetivos, emite órdenes de movimiento, configura automatizaciones sencillas y observa cómo el personaje ejecuta ataques y habilidades según sus estadísticas. La profundidad proviene de preparación, posicionamiento, selección de objetivos, equipo, atributos y talentos.

Pilares: combate deliberado con pausa · exploración compacta · builds flexibles · equipo con identidad · horror atmosférico · alcance controlado.

# 2. Visión y límites de alcance

## Pilares

- Ritmo relajado, desafío real: poca presión de reflejos; las decisiones tácticas importan.
- Exploración valiosa: pisos compactos donde encuentros, secretos y desvíos aportan valor.
- Progresión moderada: el equipo importa, sin inflación descontrolada de números.
- Libertad de builds: las clases tienen afinidades, no prohibiciones rígidas de armas o atributos.
- Low-poly HD oscuro, chunky y legible; no pixel art ni realismo fotorrealista.

## No objetivos

- No será twin-stick shooter, bullet hell, FPS, MMO ni juego como servicio.
- No tendrá árboles enormes, docenas de habilidades equipadas ni cientos de estadísticas.
- No habrá escalado automático de enemigos con el nivel del jugador.
- No será un mundo abierto continuo: se priorizan zonas y pisos diseñados con variación controlada.

Objetivo provisional: campaña principal de 10–15 horas en primera partida, 5–6 zonas principales y rejugabilidad mediante misiones secundarias variables y NG+. Son metas de planificación, no compromisos hasta probar el ritmo.

# 3. Plataforma y tecnología

Plataforma primaria: Windows PC. Teclado y ratón primero; soporte de gamepad deseable, pero puede entrar después del vertical slice.

Motor: Godot 4.x con GDScript exclusivamente; no C#. Recomendación de producción: elegir una versión estable soportada al crear el proyecto y fijarla durante cada hito. No iniciar producción en una beta ni actualizar el motor a mitad de una fase. La rama exacta debe verificarse en el archivo oficial de Godot al crear el proyecto; Godot 4.3 ya es una rama antigua para iniciar un proyecto nuevo.

- Renderizador inicial: Forward+; probar rendimiento y compatibilidad en el equipo objetivo.
- Resolución/UI escalables y opciones gráficas básicas.
- Git desde el primer día; repositorio remoto privado recomendado.
- Build ejecutable al cierre de cada hito y pruebas de regresión tras cambios importantes.

# 4. Dirección artística

Low-poly HD estilizado: geometría simplificada, volúmenes contundentes, materiales sobrios, iluminación atmosférica y efectos de alto contraste. No debe verse pixelado ni hiperrealista.

Quake y DUSK inspiran siluetas, contundencia y brutalidad visual; Half-Life y Event Horizon inspiran horror y misterio; Diablo I/II inspiran atmósfera, exploración y composición de mazmorras. No son referencias para copiar sus controles.

- Base oscura y fría con acentos energéticos controlados.
- Separar jugador, enemigos, peligros y objetos por silueta, valores y color.
- VFX legibles para impactos, estados, áreas peligrosas y barreras.
- Feedback de combate: reacción de impacto, audio por material, partículas, daño visible y gibbing selectivo cuando corresponda.

### Familias ambientales provisionales

- Complejo industrial: fábricas, almacenes, plantas eléctricas y bases militares.
- Afueras/asentamiento: calles, viviendas, granjas y edificios civiles abandonados.
- Blackwood: bosque nocturno, puestos de investigación aislados y crecimiento anómalo.
- Investigación/contención: laboratorios, hospitales y experimentación biológica.
- La Brecha: arquitectura extradimensional, estructuras orgánicas y geometría imposible.

# 5. Mundo y campaña

La amenaza es una invasión de seres de otra dimensión. Evitar un bestiario limitado a ángeles caídos o iconografía religiosa literal. Se permiten ecos cósmicos o religiosos sutiles, pero predomina el horror biológico, grotesco y extradimensional.

El jugador controla una de tres clases de exoesqueleto/mech de escala humana o ligeramente superior, de diseño atlético, militar-industrial y reconociblemente humano. El contraste entre tecnología humana y entidades inhumanas es central.

Estructura recomendada: refugio/base como hub, zonas seleccionables, pisos o misiones con objetivos claros, historia principal fija y misiones secundarias seleccionadas de módulos diseñados a mano. Hub, NPCs, acompañantes y estilo narrativo exactos siguen pendientes.

### Tamaño de pisos

Los pisos deben ser compactos, más cercanos a Diablo I que a las grandes rutas de farmeo de Diablo II. La intención es que explorar el mapa completo sea razonable y valioso, que cada punto de experiencia importe y que no exista una ruta repetible que domine la progresión. Hipótesis para prototipar: 5–10 minutos para explorar razonablemente un piso normal; niveles especiales o jefes pueden ser más largos.

No premiar exclusivamente matar todo. Distribuir valor entre encuentros, objetivos, descubrimientos, secretos y misiones para que explorar sea rentable sin obligar a limpiar cada rincón.

# 6. Clases, atributos y talentos

Clases iniciales: Vanguard (frontal/resistente), Ranger (armas balísticas/movilidad) y Aether (tecnología energética/resonancia). Nombres de trabajo.

### Identidad defensiva

- Vanguard: resistencia/absorción; posible HP temporal y pasiva de Damage Reduction ligada a Vitality u otro atributo defensivo.
- Ranger: evasión/reposicionamiento; posible Evasion y maniobra evasiva activa.
- Aether: barreras/energía; posible burbuja que absorbe daño y pasiva de resistencia o eficiencia energética.

Cada clase tendrá un árbol de talentos compacto, inspirado en la sencillez de una rama clásica de Diablo II, con activas y pasivas. Objetivo: pocas habilidades significativas, no una barra llena; 3–4 espacios activos equipados como límite

orientativo.

Atributos compartidos por todas las clases; ninguna estadística será obligatoria para que una clase funcione. Propuesta provisional: Strength (daño físico/requisitos), Agility (evasión/movilidad), Accuracy (impacto), Vitality (HP/supervivencia) y Tech (daño/eficiencia energética y requisitos). Validar si conviene mantener cinco o fusionar alguno.

Ejemplos de afinidad, no restricciones: Vanguard suele preferir Strength/Vitality y algo de Accuracy; Ranger Accuracy/Agility, con posibilidad de Strength; Aether suele preferir Tech, pero puede construir melee con Strength/Vitality. Builds no convencionales deben ser viables aunque no siempre óptimas.

Al subir de nivel se reciben puntos de atributos y de talento. Los talentos tienen requisitos de nivel y rangos. Cada partida garantiza al menos tres misiones secundarias que otorgan puntos de talento extra. Estas recompensas deben ser valiosas, pero no indispensables para una build funcional.

Respec limitado al estilo Diablo II, no libre e ilimitado. Posible servicio en la base o recompensa especial; cantidad y coste pendientes.

# 7. Combate: control y ritmo

Modelo: combate en tiempo real con pausa (RTwP), click-to-move y selección de objetivos. No es twin-stick ni requiere apuntado manual.

- Click en enemigo: seleccionar objetivo y perseguir automáticamente hasta el rango apropiado.
- Click en suelo: emitir orden de movimiento/reposicionamiento e interrumpir o reemplazar la persecución.
- Auto-ataque: continúa contra el objetivo actual mientras siga válido, incluso si el jugador no interviene o está rodeado.
- Al morir el objetivo, detenerse; no adquirir automáticamente otro enemigo por defecto.
- Las habilidades podrán configurarse Auto/Manual cuando tenga sentido. No usar pociones automáticamente por defecto.
- El jugador puede pausar para cambiar objetivo, emitir órdenes, activar habilidades manuales o usar consumibles. La pausa detiene la simulación y permite usar la UI táctica.

## Impacto y daño

Sistema híbrido de impacto: ningún ataque tiene impacto garantizado, incluso contra enemigos básicos. Accuracy aumenta la probabilidad de acertar; Evasion la reduce. Los ataques normales deben conectar habitualmente contra enemigos simples, con fallos raros y más fallos contra enemigos ágiles o defensas específicas.

Separar: Accuracy vs Evasion determina si conecta; Armor vs Armor Penetration determina cuánto daño físico atraviesa armadura; Damage Reduction (DR) reduce daño entrante mediante reglas propias. Resistencias energéticas/elementales son una capa adicional solo si aportan decisiones claras.

## Defensas, rangos y AoE

Cada clase tendrá defensas pasivas y pocas defensas activas. Las habilidades podrán ser manuales o automáticas con condiciones sencillas. Las decisiones importantes de supervivencia, como consumir pociones, son del jugador.

Cada arma y habilidad tiene su rango. El personaje busca una posición adecuada: melee se acerca, rifles mantienen distancia y tech attacks respetan su alcance. El jugador puede interrumpir el comportamiento automático con una orden de movimiento.

AoE híbrido: algunas habilidades impactan de inmediato; otras tienen wind-up, proyectil o zona persistente. Las habilidades más poderosas pueden tardar como máximo orientativo 2–3 intervalos de ataque para evitar esperas largas. Los ataques enemigos peligrosos tienen telegráficos legibles, con oportunidad razonable de reposicionarse o defenderse; no todo ataque tiene que ser esquivable.

# 8. Attack Speed, Cast Speed y breakpoints

Attack Speed (AS) controla la cadencia de ataques básicos con armas. Cast Speed (CS) controla la ejecución de habilidades con tiempo de lanzamiento. Cooldown es independiente de CS salvo que una habilidad indique lo contrario. Wind-up puede depender de CS o tener un tiempo propio.

Hipótesis de prototipo: intervalo de referencia de 1 segundo. Breakpoints compactos inspirados en Diablo II: 1/1 (1 ataque por intervalo), 3/2 (1,5 ataques por intervalo) y 2/1 (2 ataques por intervalo). Armas, equipo y talentos pueden llevar al siguiente umbral. Diferentes familias de armas pueden tener umbrales distintos.

Entre breakpoints se pueden acumular modificadores, pero la cadencia efectiva solo cambia al alcanzar el siguiente umbral. Definir un techo por tipo de arma para evitar aceleración infinita y conservar la identidad de armas lentas/pesadas. Sincronizar el impacto mecánico con la animación.

CS tendrá breakpoints propios, probablemente más conservadores. No todas las habilidades necesitan cast time; las instantáneas no deben recibir una demora artificial. Todos estos números son parámetros iniciales a validar, no balance definitivo.

# 9. Armas, equipo y loot

No restringir armas por clase. Cada clase tiene afinidades naturales, pero puede desviarse: Aether melee, Ranger con arma pesada o Vanguard con arma de fuego son posibilidades si atributos, requisitos y equipo lo permiten.

Dos conjuntos de armas al estilo Diablo II. El cambio se hace durante el combate sin pausar, con animación/demora breve; el comportamiento automático continúa con el nuevo conjunto. Habilidades dependientes del arma verifican cuál está equipada.

Slots propuestos: armas/conjuntos I y II, casco, torso, brazos, piernas y accesorios. Confirmar cantidad final. Las mejoras deben ser significativas e infrecuentes, no una lluvia de piezas casi idénticas.

Tiers tecnológicos provisionales: Standard, Military, Experimental y Relic. Calidad de objeto propuesta: normal, mágico, raro y único; revisar nombres y jerarquía. Un objeto de tier anterior de gran calidad puede superar uno de tier superior normal. Los únicos deben tener mecánicas distintivas, no solo números.

Crafting y economía simples; no hacer del crafting complejo una obligación. Durabilidad, reparación, loot compartido entre clases/campañas y comercio quedan abiertos.

# 10. Enemigos y encuentros

Bestiario de entidades extradimensionales: humanoides grotescos, bestias y criaturas extrañas. Cada enemigo necesita silueta, rol y patrón de combate legibles.

- Roles iniciales: melee agresivo, tanque blindado, hostigador a distancia, rápido/evasivo, soporte/control, criatura de área y élite.
- Composición de encuentros importa más que inflar HP.
- Feedback contundente: reacción de impacto, audio por material, partículas y gibbing selectivo.
- Armadura, resistencias, evasión y debilidades crean decisiones de equipo sin inmunidades arbitrarias.
- Jefes con fases/mecánicas legibles; NG+ añade combinaciones y mecánicas, no solo HP inflado.

Cantidad final de enemigos, gore/desmembramiento por familia y reglas de loot quedan por definir.

# 11. Misiones, proceduralidad y rejugabilidad

Generación híbrida: habitaciones y encuentros diseñados a mano; distribución, conexiones, orden y selección de side quests pueden variar proceduralmente. Evitar generar geometría arbitraria como sistema central.

Cada misión secundaria será un módulo authored con ubicación válida, condiciones, objetivo, encuentros, recompensa, historia y dependencias. Cada playthrough garantiza tres misiones secundarias con puntos de talento; el resto se sortea entre opciones compatibles.

NG+ propuesto: mayor dificultad, composiciones alteradas, misiones secundarias re-roladas, nuevos únicos/secretos y jefe/verdad final. No limitarse a multiplicadores de HP.

# 12. Controles y UX

## Esquema inicial propuesto

- Click izquierdo: seleccionar enemigo/objeto o mover según contexto.
- Click derecho: acción secundaria contextual o cancelar orden; confirmar en prototipo.
- Space o tecla configurable: pausa táctica.
- 1–4: habilidades equipadas, cada una con modo Auto/Manual configurable.
- Q/E o tecla configurable: cambiar conjunto de armas.
- Teclas configurables para inventario, personaje, mapa, misiones y consumibles.
- Gamepad: posterior; navegación UI y selección de objetivos asistida.

UI mínima: salud/recursos, objetivo y estado, rango/estado de ataque, cooldowns, conjunto de armas activo, Auto/Manual, telegráficos, mapa y feedback de loot. Evitar saturar la pantalla.

# 13. Arquitectura técnica Godot

Principio: sistemas pequeños, desacoplados y orientados a datos. Evitar un mega Player.gd y también una arquitectura empresarial innecesaria. Usar composición de nodos y Resources para contenido configurable.

- GameManager/GameState: pausa, estados globales y cambio de escenas.
- TargetingController: selección por raycast, objetivo actual y feedback.
- MovementController: click-to-move, NavigationAgent3D, persecución, rango deseado y cancelación.
- CombatController: ciclo de ataque, orden, resolución y sincronización de animación.
- Stats/Health/Defense components: estadísticas, salud, armadura, DR y resistencias.
- AbilityController: habilidades, cooldowns, cast time, Auto/Manual y condiciones.
- EquipmentComponent: weapon sets, slots y requisitos.
- EnemyAI: selección de objetivos, ataques y telegráficos.
- UI controllers: HUD, inventario, personaje, talentos, pausa y misiones.

## Resources de datos

- WeaponData: tipo, rango, velocidad base, daño, requisitos, animación, efectos y modificadores.
- AbilityData: alcance, área, cast time, cooldown, coste, modo Auto/Manual, condiciones y efectos.
- ClassData/TalentData: árbol, requisitos, rangos, pasivas y desbloqueos.
- EnemyData: stats, resistencias, rol, ataques, drops y configuración de IA.
- ItemData/AffixData: slots, tier, calidad, propiedades y reglas de generación.
- QuestData/EncounterData/RoomModuleData: contenido authored y reglas de selección/conexión.

## Estructura de carpetas sugerida

```
res://\n core/\n player/controllers/\n player/components/\n player/classes/\n combat/targeting/\n
combat/attacks/\n combat/damage/\n combat/projectiles/\n combat/effects/\n enemies/base/\n enemies/archetypes/\n
enemies/bosses/\n items/weapons/\n items/armor/\n items/consumables/\n items/affixes/\n world/hub/\n
world/zones/\n world/rooms/\n world/interactables/\n quests/\n ui/hud/\n ui/menus/\n ui/inventory/\n
ui/character/\n ui/talents/\n data/classes/\n data/enemies/\n data/weapons/\n data/abilities/\n data/items/\n
data/quests/\n art/ audio/ shaders/
```

Escenas reutilizables: Player.tscn, EnemyBase.tscn, ProjectileBase.tscn, LootDrop.tscn, RoomModule.tscn y HUD.tscn. Variantes mediante datos y escenas especializadas, no duplicando la lógica.

# 14. Plan de implementación por hitos

Construir primero un vertical slice de combate pequeño y comprobable. No empezar por la campaña completa, inventario complejo ni generación procedural amplia. Cada hito termina con una build jugable, revisión y commit/tag.

### Fase 0 — Fundación

- Crear repositorio Git y proyecto en versión estable fijada.
- Configurar inputs, resolución, renderizador y carpetas.
- Crear Main, cámara isométrica de prueba, suelo, luz y personaje placeholder.
- Documentar convenciones, README y cómo ejecutar/exportar.

Salida: proyecto abre, ejecuta y exporta una build mínima. Tag: v0.0.1-foundation.

### Fase 1 — Movimiento, cámara y selección

- Click-to-move sobre navegación.
- Selección de enemigo y feedback visual.
- Click al suelo interrumpe persecución; rangos y distancias deseadas por arma.
- Probar obstáculos y navegación con múltiples enemigos.

Salida: personaje se mueve y persigue un dummy manteniendo el rango adecuado. Tag: v0.1-targeting-movement.

### Fase 2 — Combate automático

- Ciclo de ataque por entidad y auto-ataque al objetivo seleccionado.
- Accuracy/Evasion con probabilidad de fallo no nula.
- Armor, Armor Penetration, DR y fórmula de daño separados.
- Animación sincronizada con impacto, reacción y logs de depuración.

Salida: combate 1 contra 1 y 1 contra varios sin input continuo. Tag: v0.2-auto-combat.

### Fase 3 — Pausa táctica y defensas

- Pausa de simulación con UI activa.
- Cambiar objetivo, reposicionarse, cancelar orden y usar consumibles.
- Una defensa activa por clase y pasivas sencillas.
- Probar Auto/Manual; pociones manuales por defecto.

Salida: loop RTwP controlable. Tag: v0.3-rtwp.

### Fase 4 — Armas y velocidad

- WeaponData y tres familias: melee lenta, melee rápida y rifle.
- Dos conjuntos de armas y cambio sin pausa.
- Rangos, tiempos base, AS, CS y breakpoints iniciales 1/1, 3/2 y 2/1.
- Validar identidad y sincronía de animaciones.

Salida: dos weapon sets funcionales y velocidad escalable. Tag: v0.4-weapons-speed.

### Fase 5 — Habilidades y AoE

- AbilityData, cooldown, cast time, áreas instantáneas, wind-up y zonas persistentes.
- Un AoE del Aether con preparación máxima inicial de 2–3 intervalos.
- Un ataque enemigo telegráfico y una defensa manual/automática.
- Probar pausa, alcance, línea de visión y múltiples objetivos.

Salida: habilidades diferenciadas y legibles. Tag: v0.5-abilities-aoe.

### Fase 6 — Vertical slice de combate

- Kit mínimo por clase: pasiva, defensa activa y ofensiva representativa.
- 4–6 enemigos de roles diferentes y un mini-jefe.
- Feedback de daño, resistencias, loot placeholder, VFX/SFX.
- Una arena/piso compacto con 5–10 minutos de combate/exploración.

Salida: vertical slice que demuestra el corazón del juego. Tag: v0.6-vertical-slice.

### Fase 7 — Progresión y loot

- XP, niveles, puntos de atributos/talento y requisitos.
- Árbol compacto por clase, asignación y respec limitado placeholder.
- Slots, affixes, calidades y objetos únicos de prueba.
- Dos weapon sets persistentes y guardado/carga inicial.

Salida: dos configuraciones de personaje viables. Tag: v0.7-progression-loot.

### Fase 8 — Piso y misiones

- Un piso compacto con habitaciones a mano, rutas alternativas, secreto y objetivo.
- Selección híbrida de módulos de encuentro.
- QuestData y side quests; garantizar tres misiones con recompensa de talento por partida.
- Medir duración y comprobar que no exista una ruta de XP dominante.

Salida: misión/piso completo y rejugable. Tag: v0.8-level-quest-loop.

### Fase 9 — Prototipo completo

- HUD, inventario, personaje, talentos, pausa, mapa y opciones básicas.
- VFX/SFX provisionales, iluminación, legibilidad y optimización.
- Guardar/cargar, manejo de errores, regresión y build exportada.
- Prueba externa pequeña y registro de problemas de comprensión/ritmo.

Salida: loop hub → misión → combate/loot/progresión → regreso al hub. Tag: v0.9-complete-prototype.

## 15. Git, commits y Definition of Done

### Reglas sencillas

- Un commit por unidad funcional pequeña; evitar commits gigantes.
- Prefijos: feat:, fix:, refactor:, art:, data:, docs:, test:.
- Tag al cerrar cada hito (v0.1, v0.2…).
- Antes de refactor grande o actualización de motor: commit limpio y backup/branch.
- No mezclar cambios de arte, arquitectura y balance si dificulta revertirlos.

### Definition of Done

- La escena abre sin errores ni warnings críticos.
- El flujo del hito funciona en una build ejecutable.
- Se probaron casos normales y al menos dos casos límite.
- Los valores importantes son editables desde datos/Inspector cuando corresponda.

- README/notas explican cómo probar y qué queda pendiente.
- Commit/tag creado y proyecto respaldado.

### Pruebas de combate prioritarias

- Objetivo muere mientras el jugador se aproxima: no se selecciona otro automáticamente.
- Orden de movimiento interrumpe auto-ataque/persecución.
- Enemigo sale del rango o pierde línea de visión.
- Pausa durante wind-up, cooldown o proyectil.
- Cambio de weapon set durante combate.
- Accuracy, Evasion y Armor Penetration se comportan como se espera.
- El personaje sigue actuando sin input, pero no consume pociones automáticamente.

## 16. Riesgos y decisiones abiertas

Las propuestas no deben convertirse accidentalmente en requisitos definitivos. Antes de ampliar producción, cerrar o probar:

- Nombre definitivo, historia, protagonista y estructura del hub.
- Acompañantes/party (recomendación inicial: no en el primer alcance).
- Lista final de atributos y fórmulas de hit chance, armor y DR.
- Recursos/costes de habilidades, cooldowns y reglas exactas de Auto.
- Slots finales, durabilidad, crafting, economía y loot compartido.
- Cantidad de habilidades por clase y profundidad del árbol.
- Dimensiones físicas de pisos, densidad de encuentros y XP por actividad.
- Cantidad de zonas, jefes, familias enemigas y duración final.
- Prioridad de gamepad y esquema de targeting.
- Guardado, dificultad, NG+ y persistencia de recompensas.

Regla de producción: si una decisión no es necesaria para el hito actual, se documenta como pendiente y no se construye prematuramente.

## 17. Criterios de éxito del prototipo

- Completar un piso compacto usando selección de objetivo, click-to-move y pausa.
- El combate se resuelve automáticamente sin sentirse pasivo: objetivo, posicionamiento, armas y habilidades importan.
- Ritmo deliberado, sin esperas largas ni presión excesiva de reflejos.
- AS/CS y breakpoints se sienten sin romper el balance.
- Las clases tienen identidades distintas, pero admiten builds no convencionales.
- Explorar resulta rentable sin obligar a matar todo ni repetir una ruta de XP.
- Se puede añadir contenido mediante datos/escenas sin reescribir el núcleo.

El prototipo tiene éxito cuando el loop completo se puede jugar de principio a fin y soporta expansión, no cuando contiene toda la campaña.

## Apéndice A. Glosario

- RTwP: Real Time with Pause; simulación continua que el jugador puede detener.

- AS: Attack Speed; cadencia de ataques básicos.
- CS: Cast Speed; velocidad de lanzamiento de habilidades con cast time.
- Breakpoint: umbral que cambia la cadencia efectiva.
- DR: Damage Reduction; reducción del daño recibido.
- AoE: Area of Effect; ataque que afecta una zona.
- Wind-up: preparación previa al impacto.
- Vertical slice: porción representativa y jugable del juego final.
- Data-driven: contenido definido mediante datos/Resources y no lógica específica codificada.

# Apéndice B. Referencias

Antes de fijar la versión exacta de Godot, consultar el archivo oficial y la política de soporte: https://godotengine.org/download/archive/ y https://docs.godotengine.org/en/stable/about/release_policy.html.